

Concrete FORC Lifetime Failure in CEDRUS- α : Ordinary Fresh-Message Forgery within the NGCC Chosen-Message Query Budget

Martin Feussner
Selmer Center, University of Bergen
`martin.feussner@uib.no`

3 October 2026

Provenance and review status. The candidate-specific parameter analysis, exact concentration argument, implementations, experiments, release package, and initial manuscript were produced by OpenAI Codex using Daybreak Blue at Ultra reasoning effort during an AI-run audit of NGCC signature candidates. A separately instructed hostile-review agent reconstructed the specification and attack independently, recomputed the bounds, implemented a second reduced verifier, and accepted the result only with the scope and resource qualifications stated below. The generic FORC accumulation mechanism is already described by the CEDRUS- α submission, and Abri-Katz already account for public hash trials; no novelty is claimed for either idea. At the date above, no human cryptanalyst has independently reviewed the proof, implementations, measurements, or novelty assessment. This report is a preliminary disclosure for human review and reproduction.

Abstract

CEDRUS- α is a stateless hash-based signature whose bottom few-time scheme, FORC, reveals intermediate nodes of forward hash chains. Its own Section 3.2 recognizes that signatures which reuse one bottom FORC address can accumulate enough openings to cover a new digest. We evaluate the consequence after adding the public target-hash trials that an ordinary chosen-message forger may perform.

The adversary collects independently randomized signatures and groups them by the complete h -bit bottom address. For each address, coordinate, and leaf it retains the earliest exposed chain node and that node’s authentication path. For a message never queried to the signer, it enumerates serialized public R values until $H_{\text{msg}}(R, \text{PK.seed}, \text{PK.root}, M)$ selects only covered coordinates at one retained address. Each node is advanced publicly, the coordinate openings are mixed, and one hypertree suffix for that address authenticates the resulting fixed FORC public key. This is an ordinary fresh-message forgery: it requires no reset, repeated randomness, deterministic mode, fault, related key, weak key, or implementation defect.

We derive exact first and second coverage moments. Poissonizing acquisition makes address buckets independent, so a Chebyshev bound controls the realized table rather than merely its expectation. All eight submitted parameter sets admit constant-success strategies with signing means at most $2^{77.520}$ and attacker public- H_{msg} work $Q + T$ below $2^{77.56}$, with a hard abort before 2^{80} signing requests. Here the attacker performs one H_{msg} recomputation for each acquired signature plus T target trials. For 160f, retaining one fixed six-bit address prefix gives signing mean $2^{71.458}$, fixed public-hash budget $2^{67.160}$, attacker H_{msg} work $Q + T = 2^{71.530}$, expected full-path table size $2^{77.479}$ bytes, and deterministic table capacity $2^{79.175}$ bytes. The physical qualification is substantial: expected signature traffic is $2^{85.703}$ bytes and the honest signing service performs about $2^{89.060}$ hash evaluations. Across all eight $Q + T$ -optimized rows, the largest expected resource exponent in its own

natural unit is 100.852, for honest-signer hash calls in 512s. These heterogeneous resources are not combined into one cost.

Two independent reduced implementations produced accepted fresh-message forgeries and rejected wrong-message, wrong- R , wrong-address, backward-chain, authentication-path, suffix, and whole-transcript controls. A repaired native 256f build separately confirms the normative address, PRF, WOTS, signing, and verification behavior. An all-eight-set full-parameter FORC-level experiment also confirms coordinate splicing under both endpoint conventions. It is not a full-parameter signature forgery, which was not run. The narrow new conclusion is a CEDRUS- α parameter and missing-lifetime-policy failure under the official NGCC query budget, not a new generic accumulation or public-grinding technique.

1 Introduction

CEDRUS- α is a stateless hash-based signature in the SPHINCS family [1, 2]. A hypertree authenticates the public key of a bottom few-time signature. CEDRUS- α replaces the usual FORS bottom layer with FORC, whose leaves are endpoints of short forward hash chains. A FORC signature reveals an intermediate node selected by the message digest. Later positions on the same chain become computable, so the bottom key degrades gradually as its address is reused.

This degradation is not hidden in the submission. Section 3.2 explicitly mixes over the number of signatures reaching one FORC instance and gives an approximate probability that those openings cover one new digest. Its displayed quantity has no public-hash-query variable. A forger can hold a fresh message fixed and enumerate the serialized randomizer R , which is a public input to the verifier’s message digest. Algorithm 21 reads R from the signature and recomputes the digest; it has neither `SK.prf` nor `addrnd` and makes no PRF_{msg} -origin check. The one-target probability is therefore amplified over q_H distinct public inputs. Abri and Katz give exactly this style of q_H accounting for a related hash-based construction [3, Eq. (2)]; we do not claim it as a new technique.

The concrete consequence for the submitted parameters was left unstated. The submission gives no maximum number of signatures per key, while the official NGCC evaluation model allows up to 2^{80} chosen-message signing queries [5]. Once acquisition and target grinding are optimized together, every submitted parameter set admits a fresh-message forgery below that query ceiling.

Our contributions are as follows.

1. We give an explicit coordinate-mixing forgery against the coherent specification and prove acceptance by the ordinary verifier.
2. We derive exact first and second moments of the target coverage. A Poissonized acquisition algorithm, a hard 2^{80} abort, and a concentration bound yield a fixed target-search budget and a rigorous constant success probability.
3. We evaluate all eight submitted parameter sets and keep signing queries, public-hash trials, response traffic, attacker storage and processing, and honest signing-service work as separate resources.
4. We give a six-bit address-prefix shard whose fully provisioned simple donor table is below 2^{80} bytes, plus an independent fixed-address low-memory analysis.
5. We reproduce the complete attack at reduced dimensions using ordinary hedged signatures and extensive negative controls. We separately validate a repaired native 256f core and an all-eight-set, full-parameter FORC splice, excluding the known submitted source deviations while avoiding any claim of a full-size forgery run.

6. We delimit novelty to the exact CEDRUS- α parameter/lifetime consequence. FORC accumulation and the q_H multiplier are prior art.

The result is a specification-level parameter and key-lifetime failure in the ideal-hash model. It is not machine-practical: the signing-query and public- H_{msg} counts sit beside very large responses and very expensive honest signing. We report those resources separately and make no claim that the full-parameter attack was executed on the audit machine.

2 Relevant construction

2.1 Keys, randomized signing, and digest parsing

For node length n , key generation produces

$$\text{SK} = (\text{SK.seed}, \text{SK.prf}, \text{PK.seed}, \text{PK.root}), \quad \text{PK} = (\text{PK.seed}, \text{PK.root}). \quad (1)$$

The first three components are independently sampled n -byte strings, and PK.root is the hypertree root.

Ordinary hedged signing samples a fresh n -byte addrnd and computes

$$R = \text{PRF}_{\text{msg}}(\text{SK.prf}, \text{addrnd}, M), \quad (2)$$

$$\text{digest} = H_{\text{msg}}(R, \text{PK.seed}, \text{PK.root}, M). \quad (3)$$

The digest parses as k FORC pairs $(x_i, \ell_i) \in [0, 2^a) \times [0, W)$, a bottom-tree index idxTree of $h - h_0$ bits, and a bottom-leaf index idxLeaf of h_0 bits, where $W = w'$. We write

$$A = (\text{idxTree}, \text{idxLeaf}) \quad (4)$$

for the complete h -bit bottom FORC address. The verifier receives R in the signature: Algorithm 21 applies $\text{SIG.getR}()$ and recomputes (3). Forgery validity is determined by this public verification algorithm. It has no secret input with which to test relation (2), so any serialized n -byte R^* is syntactically eligible and the attacker may grind distinct R^* values on the fresh target message.

Table 1 gives the parameters used below. The final column is the specification’s approximate number of hash evaluations for one honest signature, used only to expose signing-service cost.

Table 1: Submitted CEDRUS- α parameters.

set	n bytes	h	d	a	k	W	OTS len	$ \sigma $ bytes	sign hashes
160s	20	68	9	10	16	8	30	10,300	2,743,319
160f	20	66	17	7	28	4	40	19,420	199,011
256s	32	67	9	12	23	4	48	25,568	4,163,230
256f	32	65	14	8	45	2	63	43,296	456,199
384s	48	65	8	12	38	4	88	60,672	5,347,900
384f	48	65	12	10	51	2	80	76,176	1,490,831
512s	64	65	8	13	47	4	101	98,048	10,555,995
512f	64	66	11	10	66	4	109	127,488	2,481,272

2.2 FORC equations

Put $B = 2^a$. For coordinate i , leaf x , and chain position j , let the global chain index be $u = iB + x$. At complete address A , the coherent Algorithms 13–18 define

$$s_{i,x} = \text{PRF}(\text{PK.seed}, \text{SK.seed}, \text{ADRS}_{\text{FTS-PRF}}(A, u)), \quad (5)$$

$$C_{i,x}(0) = s_{i,x},$$

$$C_{i,x}(j+1) = F(\text{PK.seed}, \text{ADRS}_{\text{FTS-CHAIN}}(A, u, j+1), C_{i,x}(j)), \quad (6)$$

$$L_{i,x} = C_{i,x}(W). \quad (7)$$

The B leaves $L_{i,x}$ form a height- a Merkle tree with root T_i . The FORC public key is

$$\text{PK}_{\text{FORC}}(A) = T_k(\text{PK.seed}, \text{ADRS}_{\text{FTS-ROOT}}(A), T_0 \parallel \cdots \parallel T_{k-1}). \quad (8)$$

For digest coordinate (x_i, ℓ_i) , the signature contains $C_{i,x_i}(\ell_i)$ and the a Merkle siblings for leaf x_i . Verification applies $W - \ell_i$ public forward steps, reconstructs T_i with that path, and evaluates (8). A hypertree suffix signs $\text{PK}_{\text{FORC}}(A)$ at the same complete address and authenticates it to PK.root .

2.3 Resolution of specification ambiguities

The PDF contains stale fields and names, but no ambiguity supplies the attack. We use the following coherent reading.

1. Equation (1.3), the H_{msg} rows of Tables 1.2–1.3, and Figure 1.14 retain a dangling public-counter formalism. The exact token ctrFTS occurs only in Algorithm 20, line 16, where an undefined value is appended. Algorithm 20, line 4 computes H_{msg} without it; Algorithm 21 neither parses nor checks it, computes H_{msg} without it, and gives a counter-free length equation. Table 1.1 and Equation (4.1) give the same counter-free sizes. No generation loop, acceptance predicate, rejection rule, or counter distribution is specified. We therefore follow the executable, counter-free Algorithms 20–21 and published sizes. An unconstrained public counter would add another public grinding input and four response bytes; it would not defeat the attack.
2. Stale FORS and PORS names in Algorithms 20–21 mean the FORC algorithms defined in Section 1.9.
3. The preliminary digest-length formula omits the $k \log_2 W$ position bits. The later formula, parser, and Algorithms 16–21 include them.
4. Algorithms 14–18 operationally disclose positions $0, \dots, W - 1$ and authenticate $C(W)$, while the prose can be read as authenticating $C(W - 1)$. Write $E \in \{W - 1, W\}$ for the chosen endpoint. In either reading, a donor can reach a target exactly when $\ell_{\text{donor}} \leq \ell_{\text{target}}$: it advances by their difference and verification applies the remaining $E - \ell_{\text{target}}$ steps. Relabelling positions by one also preserves this order. Consequently g_r , the pairwise second moment, and every concrete table below are invariant under the endpoint convention.

Reading the stale counter as evidence for an unpublished forced-pruning rule would define a different scheme. Ordinary ideal- H_{msg} chunks are uniform and independent in the submitted algorithms. All sixteen reference and optimized source trees corroborate the counter-free reading: signing serializes only R before the FORC bytes, verification consumes that R , the message hash input is $R \parallel \text{PK} \parallel M$, and SPX_BYTES contains no four-byte counter term.

3 The coordinate-mixing forgery

Lemma 1 (Forward completion). *Fix a key, complete address A , coordinate i , and leaf x . An opening at position ℓ constructs a verifier-valid opening for every target position $\ell' \geq \ell$ at the same (A, i, x) .*

Proof. Apply (6) exactly $\ell' - \ell$ times to obtain $C_{i,x}(\ell')$. Keep the donor's authentication path. The verifier applies the remaining $E - \ell'$ steps and reaches the same fixed endpoint $C_{i,x}(E)$, where $E = W$ under Algorithms 14–18 and $E = W - 1$ under the shifted prose reading. It therefore reconstructs the same leaf and root T_i in either convention. $\square$

Lemma 2 (Coordinate mixing). *At a fixed complete address A , valid openings for different coordinates may be taken from different signatures and combined with one donor hypertree suffix.*

Proof. For a fixed key and A , each T_i in (8) is fixed independently of the digest that selected an opening. Every valid coordinate opening reconstructs its fixed T_i , so their ordered compression is the fixed $\text{PK}_{\text{FORC}}(A)$. Every honest suffix at A authenticates exactly that value. The proof never transplants a path across leaves, coordinates, or addresses. $\square$

The adversary runs the following algorithm.

1. Query ordinary randomized signatures on distinct chosen messages. Recompute their public digests and group them by the complete address A .
2. For every observed (A, i, x) , retain the donor with the smallest ℓ , its node, and its complete authentication path. Retain one hypertree suffix for each occupied address.
3. Choose a message M^* never sent to the signing oracle. Enumerate distinct n -byte strings R^* and evaluate $H_{\text{msg}}(R^*, \text{PK.seed}, \text{PK.root}, M^*)$.
4. Stop when its address A^* is retained and every selected (x_i^*, ℓ_i^*) has a donor at (A^*, i, x_i^*) with $\ell_i \leq \ell_i^*$.
5. Advance every donor to its target position, copy its path, and attach one retained suffix for A^* .

Theorem 3 (Fresh-message acceptance). *The output of the algorithm is accepted by the specification-conformant verifier on M^* .*

Proof. The verifier recomputes the target digest from the serialized R^* and fresh M^* . The first lemma shows that each constructed coordinate reaches the correct fixed endpoint and root. The second lemma shows that their compression is the fixed $\text{PK}_{\text{FORC}}(A^*)$ and that the copied suffix authenticates it to the public root. All verifier checks therefore pass. Freshness follows because M^* was never queried; R^* need not satisfy the signer's secret PRF_{msg} relation. $\square$

Acquisition uses independent fresh signer randomness. The attack does not ask for, cause, or test an R collision. Complete messages and complete FORC vectors need not repeat.

4 Exact coverage and concentration

4.1 One address

Suppose r signatures reach one address. For one target coordinate with position y , a uniform donor covers it when it chooses the same leaf and one of positions $0, \dots, y$, with probability $(y + 1)/(BW)$. Averaging over a uniform target position gives the exact mean covered fraction

$$g_r = \frac{1}{W} \sum_{y=0}^{W-1} \left[1 - \left(1 - \frac{y+1}{BW} \right)^r \right]. \quad (9)$$

The k coordinate streams are independent conditional on r , so the mean covered full-vector fraction is g_r^k .

Concentration across addresses needs a second moment. Let C_r be the covered fraction of one coordinate's BW -element target domain, and define $m_r(s) = (1 - s/(BW))^r$. For two ordered targets at positions y, z , the probability that both donor sets have been hit is

$$\begin{cases} 1 - m_r(y+1) - m_r(z+1) + m_r(\max\{y+1, z+1\}), & x = x', \\ 1 - m_r(y+1) - m_r(z+1) + m_r(y+z+2), & x \neq x'. \end{cases} \quad (10)$$

Averaging (10) over the B^2W^2 ordered pairs gives $s_r = \mathbb{E}[C_r^2]$. Independence across coordinates gives second moment s_r^k for the full-vector covered fraction.

4.2 All addresses

Let the adversary first sample its acquisition count $L \leftarrow \text{Pois}(Q)$ and abort if $L > 2^{80}$. Under the ideal digest model, Poisson splitting makes the 2^h complete-address loads independent $\text{Pois}(\lambda)$ variables, where $\lambda = Q/2^h$. Define

$$\mu = \mathbb{E}_{S \leftarrow \text{Pois}(\lambda)}[g_S^k], \quad \nu = \mathbb{E}_{S \leftarrow \text{Pois}(\lambda)}[s_S^k]. \quad (11)$$

For realized tables Z_A , one public target succeeds with probability

$$P = 2^{-h} \sum_A Z_A. \quad (12)$$

Independence of the buckets and $0 \leq Z_A \leq 1$ give

$$\mathbb{E}[P] = \mu, \quad \frac{\text{Var}(P)}{\mathbb{E}[P]^2} \leq \frac{\nu}{2^h \mu^2}. \quad (13)$$

The numerical sums are truncated after a recorded Poisson Chernoff tail below 2^{-299} ; that tail is added to the second-moment upper bound.

Chebyshev's inequality yields

$$\Pr[P < \mu/2] \leq \frac{4\nu}{2^h \mu^2}. \quad (14)$$

On the complementary good-table event, the fixed budget

$$T = \left\lceil \frac{2}{\mu} \right\rceil \quad (15)$$

succeeds with probability at least $1 - e^{-1}$. Subtracting the signing-cap abort probability gives a rigorous overall lower bound. This is a statement about the realized table, not only expected coverage.

4.3 Address-prefix sharding

Fix a b -bit prefix before acquisition and retain only addresses with that prefix. The retained bucket loads remain independent with the same λ . The success mean becomes $2^{-b}\mu$, the number of stored buckets falls by 2^b , and the relative-variance bound in (13) grows by 2^b . The attack then uses $T_b = \lceil 2^{b+1}/\mu \rceil$. This gives a direct time-memory tradeoff without replaying randomness or predicting acquisition addresses.

5 Concrete results

5.1 All eight submitted sets

Table 2 minimizes $Q + \lceil 2/\mu \rceil$. Each exponent is base two. Q is the signing-query Poisson mean and also the number of attacker H_{msg} recomputations needed to recover the acquired signatures’ addresses; the actual signing count is hard-capped by aborting before 2^{80} . T is a fixed fresh-target H_{msg} budget. Thus $Q + T$ is attacker public- H_{msg} work, not a sum of heterogeneous oracle costs. Every success lower bound exceeds 0.6321205588 after subtracting the displayed table tail and the negligible cap-abort probability.

Table 2: Rigorous all-address acquisition and target-search results.

set	signer queries $\log_2 Q$	target H_{msg} $\log_2 T$	attacker H_{msg} $\log_2(Q + T)$	bad-table bound
160s	74.415	70.593	74.514	$< 2^{-62.89}$
160f	71.165	66.878	71.237	$< 2^{-53.07}$
256s	76.670	72.237	76.735	$< 2^{-64.23}$
256f	72.140	67.248	72.188	$< 2^{-54.93}$
384s	76.100	71.159	76.146	$< 2^{-62.38}$
384f	74.430	69.213	74.468	$< 2^{-60.30}$
512s	77.520	72.367	77.560	$< 2^{-62.68}$
512f	76.145	70.790	76.180	$< 2^{-61.81}$

The 160f row’s $2^{71.237}$ figure is attacker public- H_{msg} work $Q + T$; the signing-oracle count is the separate $Q = 2^{71.165}$ and the fresh-target budget is $T = 2^{66.878}$. Table 3 reports four separate physical-resource categories at the same optima. The table payload uses a deliberately simple sufficient record: one chain node, one position byte, and all a authentication nodes for every observed (A, i, x) , plus one complete hypertree suffix per occupied address. It assumes no Merkle multiproof compression. Response and service figures are expected costs at the listed Poisson means; the donor-table column is expected payload, exclusive of allocator and database overhead.

Table 3: Physical resources at the all-address $Q + T$ optima (base-two exponents).

set	response bytes / bits	donor bytes	record updates	signer hash calls
160s	87.745 / 90.745	86.177	78.415	95.802
160f	85.410 / 88.410	83.251	75.972	88.767
256s	91.312 / 94.312	89.760	81.194	98.659
256f	87.542 / 90.542	85.458	77.632	90.939
384s	91.989 / 94.989	90.268	81.348	98.451
384f	90.647 / 93.647	88.696	80.102	94.938
512s	94.101 / 97.101	92.398	83.075	100.852
512f	93.105 / 96.105	90.930	82.189	97.388

Thus the largest expected exponent in Table 3, in its own natural unit, is $2^{100.852}$ honest-service hash calls for 512s. The maxima for returned data, donor payload, and record updates are separately $2^{97.101}$ bits, $2^{92.398}$ bytes, and $2^{83.075}$ updates. The largest Q and T exponents in Table 2 are 77.520 and 72.367. These numbers are neither added nor compared as interchangeable units.

For 160f, a hard worst case after the 2^{80} abort contains at most $2^{94.245}$ returned signature bytes and approximately $2^{97.603}$ PDF-counted honest-signer hash calls. These worst-case caps are distinct from the expectations in Table 3.

5.2 A fully provisioned table below 2^{80} bytes

The six-bit 160f shard gives the cleanest bounded-memory construction near the requested resource band. For one donor record of

$$((a + 1)n + 1) \text{ bytes} \quad (16)$$

and one address suffix of $(d \text{ len} + h)n + 16$ bytes, a table with every slot provisioned has deterministic payload

$$2^{h-6} [k2^a((a + 1)n + 1) + (d \text{ len} + h)n + 16]. \quad (17)$$

Table 4 reports both this capacity and the expected occupancy.

Table 4: CEDRUS- α -160f six-bit-prefix shard.

resource	base-two exponent or value
signing-query mean Q	71.458
fixed public- H_{msg} budget T_6	67.160
attacker H_{msg} calls $Q + T_6$	71.530
expected full-path donor-table bytes	77.479
deterministic fully provisioned payload bytes	79.175
returned signature bytes / bits	85.703 / 88.703
honest signing-service hash calls	89.060
Chebyshev bad-table bound	$< 2^{-48.61}$
overall success lower bound	> 0.6321205588

The expected returned traffic and signer work do not fall with sharding: nonretained responses must still be obtained and their R values hashed to learn their addresses. The deterministic memory figure is a payload bound, not a claim that an implementation can allocate $2^{79.175}$ bytes on present hardware.

5.3 Alternative physical objectives and fixed-address memory

Different cost objectives choose different 160f points. Minimizing the larger of returned *bits* and public- H_{msg} evaluations gives $\log_2 Q = 70.085$, $\log_2 T = 87.269$, $2^{87.330}$ response bits, $2^{82.402}$ expected full-path table bytes, and $2^{87.687}$ honest-service hashes. Balancing public hashes against signer hashes instead gives $\log_2 Q = 70.065$, $\log_2 T = 87.631$, $2^{87.310}$ response bits, and $2^{87.667}$ signer hashes. These are alternative tradeoffs, not resources paid simultaneously at separate optima. Each shows why no single “complexity” number describes the attack.

A low-memory independent strategy preselects one complete 160f address. With $r = 554$ retained hits, Paley–Zygmund gives probability at least 0.247483 that the table covers at least half its mean target mass. Using $\lceil 2r2^{66} \rceil$ signing queries gives exponent 76.113742 and a Chernoff hit-collection failure below $2^{-199.81}$. A fixed $2^{72.445408}$ public-target budget yields attacker H_{msg} exponent 76.222974 and success at least 0.156439 minus that failure. The fully provisioned one-address payload is only $2^{19.175}$ bytes, but returned traffic and signer work rise to approximately $2^{90.359}$ bytes and $2^{93.716}$ hashes.

6 End-to-end validation

6.1 Independent reduced verifier

The hostile-review implementation uses $h = 4$, $a = 2$, $k = 4$, $W = 4$, and 16-byte nodes. It independently implements domain-separated FORC chains, complete addresses, leaf-specific

authentication paths, an address-fixed FORC public key, attacker-controlled serialized R , ordinary hedged signing, and a reduced Merkle-authenticated suffix. SHAKE is an ideal-primitive stand-in at the reduced node size.

Its fixed run obtained 26 ordinary randomized signatures and forged on a never-queried message using donors from three distinct messages. The verifier accepted the result. It rejected the same bytes after separately changing the message, R , a FORC authentication node, the hypertree suffix, the chain direction, or the complete address.

A second, fuller reduced hypertree implementation independently produced both a one-transcript forward forgery and a coordinate-mixed forgery. The latter used two donor transcripts, was accepted on a fresh message, and explicitly checked that no one acquired transcript covered the target. Its unadvanced node, backward-chain, path-tamper, wrong-address, and cross-layer-suffix controls all rejected.

These are complete reduced experiments, not extrapolated full-parameter runs. They validate the algebraic splice and all verifier bindings that the proof uses.

6.2 Full-parameter FORC-level splice

A separate deterministic experiment used the exact submitted (n, h, a, k, W) values, full complete-address widths, Algorithm 16 bit order, and Algorithms 13–18 address dependencies for all eight parameter sets. At one complete address per set it constructed every leaf of every FORC tree, formed three valid FORC digest openings, selected target coordinates from all three donors, advanced the exposed nodes, and reused their own authentication paths. For every set the mixed opening reconstructed the same fixed FORC public key; no complete donor vector covered the target. Authentication-path, unadvanced-node, and wrong-complete-address controls reconstructed a different key and were rejected.

The experiment was run twice per set: once with the Algorithms 14–18 endpoint $C(W)$ and once with the prose-shifted endpoint $C(W - 1)$. Every run passed, confirming computationally that only the order $\ell_{\text{donor}} \leq \ell_{\text{target}}$ matters. SHAKE256 was a deterministic ideal-primitive stand-in; the experiment validates full submitted FORC dimensions and the structural splice, not an SM3 implementation and not a full CEDRUS- α forgery. It performs neither the predicted acquisition nor the public target grind.

6.3 Repaired native 256f conformance

The submitted source contains three public deviations: 160-bit WOTS message truncation, eight-bit FORC chain-address truncation, and hash/PRF input differences [7]. None is used here. We repaired a fresh 256f reference tree to the PDF’s full-width address fields, Table 1.2 PRF/hash input organization, and full constant-sum WOTS message handling.

The native run generated two ordinary hedged signatures on one message and confirmed that their R values differ. It checked the published 43,296-byte signature size; ordinary verification; rejection after changing the message, R , a FORC node, or a WOTS node; rejection of truncated and extended signatures; rejection under an independent key; a full-width chain-address test; a fixed Table 1.2 PRF vector; and full-message constant-sum WOTS use. Every check passed.

The native run is a conformance check, not a full-size forgery. The full attack would require the resources in Section 5 and was not executed.

7 Evaluation boundary and security impact

The official NGCC Evaluation Criteria, Section 1(2), allow a chosen-message attacker no more than 2^{80} signing queries [5]. The separate submission requirements, Section 2(2), require a

signature key pair to support at least 2^{64} different messages [6]. The latter is a minimum support requirement, not an implicit universal maximum.

A submission-wide review found no maximum signatures per key, mandatory key-rotation rule, or $2^{64}/2^{80}$ lifetime cap. The reviewed material was the 53-page specification, both two-page English and Chinese basic-information forms, all sixteen reference/optimized implementation READMEs, all sixteen source trees, and the specification’s security-claim chapter. The forms contain only identity/contact fields; the byte-identical READMEs contain generic interface, harness, use, and build notes but no lifecycle rule. Chapter 3, Section 3.2 instead says “let q_{sig} be the number of signing queries overall,” uses it as an unconstrained variable in Equations (3.1)–(3.2), and sums through $q = q_{\text{sig}}$. The source’s 64-bit condition is only an index-width compile guard, not a usage rule. The attack samples its query count but aborts before request 2^{80} ; it therefore respects the evaluation ceiling in every execution. For the 160f six-bit shard, the Chernoff upper bound on that abort event is smaller than $2^{-8.5 \cdot 10^{24}}$.

The authors’ related framework paper makes reduced capacity explicit when it is intended: its Section 5.5 is titled “CEDRUS Supporting Less than 2^{64} Signatures” and gives separate CEDRUS+/CEDRUS+FP examples with stated maximum capacities [2]. Its ordinary CEDRUS- α treatment in Section 5.2 and Appendix R.3 instead retains a q_S -parameterized EU-CMA reduction without imposing a cap on the NGCC sets. Those framework parameter sets are distinct, so this is corroborating documentation practice, not a claim that Section 5.5 normatively governs the submission.

The distinction is important. At exactly $Q = 2^{64}$, 160f’s mean one-target success is $2^{-161.7365}$. This method does not break a deployment that explicitly and enforceably retires each key after 2^{64} signatures. The submitted construction contains no such rule. The result is therefore a parameter and absent-lifetime-policy failure within the stated evaluation model, rather than a claim that the generic FORC design is unsalvageable.

The impact is an existential fresh-message signature accepted by the ordinary verifier, not merely a reduced security estimate. It neither recovers `SK.seed` nor needs to: the accumulated openings and public grind directly create a new signature. The attack is unconditional with respect to signing state and key choice, while retaining the ideal-primitive assumptions under which the scheme’s concrete analysis is stated.

8 Scope, limitations, and repair

The principal limitations are quantitative.

- No full-parameter forgery was run. The formal query counts represent astronomically more data and service work than the NREC audit host can instantiate.
- The probability theorem uses the specification’s ideal independent H_{msg} model. It is not an empirical statement about bias in one fixed SM3 instantiation.
- The simple memory model counts cryptographic payload and explicit address metadata. Real storage systems add indexing and allocator overhead.
- The attack exceeds 2^{64} signing queries. A lower explicit and enforced key lifetime can exclude it.

All acquisition queries are ordinary independently randomized chosen-message signatures. The construction uses no reset, repeated randomizer, deterministic mode, related key, weak key, fault, or implementation deviation. Its material scope condition is the absence of a lower enforced per-key lifetime: a deployment that retires the key after 2^{64} signatures falls outside the range established here.

The most direct repair is to state and enforce a per-key signature limit whose value is included in the concrete security claim. A 2^{64} limit defeats this route for the submitted rows,

although a revised analysis must still include all public-hash queries and multi-user targets. Alternatively, increase the complete address and FORC parameters so the exact quantity

$$1 - (1 - p_{\text{one}}(q_S))^{q_H} \quad (18)$$

remains below the target advantage for the claimed (q_S, q_H) range. The first and second moments above provide a conservative way to evaluate the realized multi-address table.

A stateful signer could allocate bottom addresses without reuse, but that changes the stateless design and requires rollback-safe persistent state. Merely serializing a public counter does not help if the attacker can vary it or R during the public target search. Any forced-pruning or rejection rule would need a new byte-level specification, distribution analysis, and proof; none is present in the submitted algorithms.

9 Prior art and novelty boundary

The closest prior art is the candidate specification itself. CEDRUS- α Section 3.2, Equations (3.1)–(3.2), mixes over address occupancy and gives an approximate FORC completion probability for one fresh target [1]. It already describes the accumulation mechanism. The broader CEDRUS framework retains FORC coverage and interleaved target-subset resilience in its security discussion [2]. The SPHINCS+ framework likewise treats degradation across reused few-time instances [4].

Abri and Katz’s forced-pruning analysis is even closer to the missing concrete step. Its Equation (2) multiplies a one-target success term by the number of public hash trials $q_{H,2}$ after mixing over bottom-instance occupancy [3]. This is direct prior art for offline public-hash amplification. We claim neither that multiplier nor conditioning-safe coverage counting as new.

At the 3 October 2026 checkpoint, the public NGCC page listed three CEDRUS- α implementation findings and no parameter/lifetime forgery [7]. This is a best-effort public-state statement rather than a proof that no unpublished analysis exists.

The narrow contribution of this report is the candidate-specific conclusion: exact chain-position coverage and second moments for all eight submitted sets, a concentration-controlled coordinate-mixing forgery, optimized q_S/q_H tradeoffs with complete physical resource accounting, and the observation that the absence of a specified per-key lifetime cap places the attack inside the official NGCC chosen-message query budget.

10 Conclusion

Ordinary independently randomized CEDRUS- α signatures can be accumulated by their complete bottom FORC address. An attacker who retains the earliest node and path for each coordinate and leaf can combine openings from different messages. Public grinding of serialized R on a fresh message finds a digest covered by the accumulated table, and one address-matched hypertree suffix completes a signature accepted by the ordinary verifier.

Exact second moments show that the all-address table concentrates, while a hard abort keeps signing queries within 2^{80} . All eight parameter sets use fewer than $2^{77.56}$ attacker public- H_{msg} calls $Q + T$, with Q signing queries reported separately. The 160f six-bit shard gives a deterministic donor payload below 2^{80} bytes, but still requires about $2^{85.70}$ response bytes and $2^{89.06}$ honest-service hash calls. At the all-address optima, 512s reaches the largest expected natural unit exponent, $2^{100.852}$ honest-signer hashes. The attack is therefore a concrete parameter/lifetime failure in the evaluation model, not a machine-practical computation.

CEDRUS- α already identifies FORC accumulation, and Abri–Katz already identify the public-query multiplier. The repair is to make the claimed key lifetime and public-hash budget explicit and choose or enforce parameters for their joint effect.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

Artifact availability

The accompanying reproducer contains both reduced verifiers, the exact all-parameter probability and resource calculation, the all-eight-set full-parameter FORC splice, a bundled repaired 256f native source tree, the submission-wide lifetime/counter audit, negative controls, expected machine-readable outputs, and a one-command replay. It requires no network access. The artifact verifies its immutable inputs before running and confines generated files to disposable build and work directories.

References

- [1] Siwei Sun, Zhen Qin, Fangyu Zheng, and Zhiyu Zhang, *CEDRUS α : A Stateless Hash-based Signature Scheme*, NGCC submission specification, 29 June 2026.
- [2] Zhen Qin, Siwei Sun, and Fangyu Zheng, *Extending the SPHINCS+ Framework: Varying the Tree Heights and Chain Lengths*, IACR Cryptology ePrint Archive, Report 2025/2236, revised 6 September 2026, <https://eprint.iacr.org/2025/2236>.
- [3] Mehdi Abri and Jonathan Katz, *Shorter Hash-Based Signatures Using Forced Pruning*, IACR Cryptology ePrint Archive, Report 2025/2069, revised 2026; CRYPTO 2026, <https://eprint.iacr.org/2025/2069>.
- [4] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe, *The SPHINCS+ Signature Framework*, IACR Cryptology ePrint Archive, Report 2019/1086, 2019, <https://eprint.iacr.org/2019/1086>.
- [5] National Cryptography Administration of China, *Evaluation Criteria for Candidate Algorithms of the Next-Generation Public-Key Cryptography Competition*, Section 1(2), 2025, <https://www.niccs.org.cn/niccs/Notice/1975896137741635584/tT7TSQiz.pdf>.
- [6] National Cryptography Administration of China, *Submission Requirements for the Next-Generation Public-Key Cryptography Competition*, Section 2(2), 2025, <https://www.niccs.org.cn/niccs/Notice/1Dop1mav.pdf>.
- [7] NGCC Security Reports, *CEDRUS- α (sign-04)*, <https://ngcc.dev/reports/sign-04.html>, accessed 3 October 2026.